

Obsługa ROS Topics

Wprowadzenie - polecenia w terminalu

ROS Topics używane są do komunikacji rozgłoszeniowej. Nie ma znaczenia kto jest nadawcą, a kto odbiorcą wiadomości. Za kodowanie przesyłanej wiadomości odpowiada **Publisher**, a za rozkodowanie **Subscriber**.

Typy wiadomości w paczce przechowywane są w katalogu msg, a rozszerzenie wiadomości to .msg. Przykładowa nazwa pliku z wiadomością: **PoseStamped.msg**. **PoseStamped** wskazuje nazwę typu wiadomości, który nie zawiera .msg.

Na topicu o podanej nazwie obsługiwany jest tylko jeden konkretny typ wiadomości. Nie można mieszać typów wiadomości.

Struktura wiadomości

typ_pola nazwa_pola

Z lewej strony należy podać typ pola, a z prawej nazwę pola. Typ pola może być prosty (np. bool, int8, uint16, float32, string, time) lub złożony. Typ pola złożony wykorzystuje wiadomości pochodzące z innych paczek ROS. Można go rozpoznać po następującym zapisie:

nazwa_paczki/nazwa_typu_wiadomosci_ros nazwa_pola

rosmmsg - komenda do obsługi wiadomości (MSG)

```
In [1]: # --help - wyświetla pomoc do dowolnego polecenia
!rosmmsg --help
```

rosmmsg is a command-line tool for displaying information about ROS Message types.

Commands:

rosmmsg show	Show message description
rosmmsg info	Alias for rosmmsg show
rosmmsg list	List all messages
rosmmsg md5	Display message md5sum
rosmmsg package	List messages in a package
rosmmsg packages	List packages that contain messages

Type rosmmsg <command> -h for more detailed usage

Ogólny sposób wyświetlania informacji ze strukturą wiadomości w ROS:

rosmmsg show *nazwa_paczki/nazwa_typu_wiadomosci_ros*

Wyświetlenie przykładowej struktury wiadomości typu **RobotInfo** znajdującej się w paczce **pkg_tsr**

```
In [2]: !rosmmsg show pkg_tsr/RobotInfo
```

```
int32 robot_id
string info
```

Wyświetlenie przykładowej struktury wiadomości typu **PoseStamped** znajdującej się w paczce **geometry_msgs**

```
In [3]: !rosmmsg info geometry_msgs/PoseStamped
```

```
std_msgs/Header header
  uint32 seq
  time stamp
  string frame_id
geometry_msgs/Pose pose
  geometry_msgs/Point position
    float64 x
    float64 y
    float64 z
  geometry_msgs/Quaternion orientation
    float64 x
    float64 y
    float64 z
    float64 w
```

Dla wiadomości typu **PoseStamped**, aby dostać się do pól z informacją o położeniu **x** należy dostać się kolejno przez pola **pose.position.x**.

Sprawdzenie struktury wiadomości jest możliwe po przekazaniu nazwy samego typu wiadomości bez wskazania paczki, z której dany typ wiadomości pochodzi.

rosmg show *typ_wiadomości_ros*

Uwaga

Użycie samej nazwy typu wiadomości bez wskazania paczki może spowodować wyświetlenie większej liczby wiadomości. Dla przykładu wiadomość typu **Pose** znajduje się jednocześnie w paczce **geometry_msgs** i **turtlesim**. W zależności od paczki wiadomości różnią się między sobą strukturą i nie są ze sobą zgodne pomimo tej samej nazwy typu.

Dla komunikacji **ROS Topics** typ wiadomości pomiędzy Publisherem, a Subscriberem musi być zgodny zarówno co do nazwy typu jak i paczki, z której pochodzi.

```
In [4]: !rosmg show Pose
```

```
[geometry_msgs/Pose]:
geometry_msgs/Point position
  float64 x
  float64 y
  float64 z
geometry_msgs/Quaternion orientation
  float64 x
  float64 y
  float64 z
  float64 w

[turtlesim/Pose]:
float32 x
float32 y
float32 theta
float32 linear_velocity
float32 angular_velocity
```

rostopic - komenda do obsługi ROS Topics

```
In [5]: !rostopic --help
```

rostopic is a command-line tool for printing information about ROS Topics.

Commands:

rostopic bw	display bandwidth used by topic
rostopic delay	display delay of topic from timestamp in header
rostopic echo	print messages to screen
rostopic find	find topics by type
rostopic hz	display publishing rate of topic
rostopic info	print information about active topic
rostopic list	list active topics
rostopic pub	publish data to topic
rostopic type	print topic or field type

Type rostopic <command> -h for more detailed usage, e.g. 'rostopic echo -h'

Wyświetlenie listy aktualnie dostępnych topic.

```
In [6]: !rostopic list
```

```
/rosout
/rosout_agg
/turtle1/cmd_vel
/turtle1/color_sensor
/turtle1/pose
```

Dla aktywnego topicu o podanej nazwie wyświetla informację o typie wiadomości, aktualnych nazwach node'ów, które publikują i subskrybują podany topic.

rostopic info *nazwa_topic*

```
In [7]: !rostopic info /turtle1/cmd_vel
```

Type: geometry_msgs/Twist

Publishers: None

Subscribers:

* /turtlesim (http://localhost:36109/)

Dla aktywnego topicu o podanej nazwie wyświetla informację tylko o typie wiadomości.

rostopic type *nazwa_topic*

```
In [8]: !rostopic type /turtle1/cmd_vel
```

geometry_msgs/Twist

Subskrypcja danych przychodzących na danym topicu. Dla ciągłego odczytu należy zastosować polecenie:

rostopic echo *nazwa_topic*

Poniższy przykład zawiera dodatkowy parametr **-n 1**, który pozwala odczytać tylko jedną wiadomość. Podając tylko nazwę topicu odczytywane są wszystkie pola z wiadomości.

```
In [9]: !rostopic echo -n 1 /turtle1/pose
```

```
x: 5.544444561004639
y: 5.544444561004639
theta: 0.0
linear_velocity: 0.0
angular_velocity: 0.0
---
```

Możliwe jest odczytanie wartości z pojedynczego pola wiadomości. Dla wiadomości na topicu **/turtle1/pose** do odczytania wartości o orientacji robota wystarczy po / wskazać nazwę pola wiadomości **theta**.

```
In [10]: !rostopic echo -n 1 /turtle1/pose/theta
```

```
0.0
---
```

Opublikowanie wiadomości z poziomu terminala.

!(polecenie w terminalu)

rostopic pub --once *nazwa_topic* wiadomość

Parametr **--once** dla polecenia **rostopic pub** powoduje wysłanie wiadomości tylko raz. Znak \ w poniższej komendzie oznacza przejście do nowej linii. Bez tego znaku całe polecenie powinno być zapisane w jednej linii.

```
In [11]: !(rostopic pub --once /turtle1/cmd_vel geometry_msgs/Twist "{linear: {x: 2.0, y: 0.0, z: 0.0}, \
angular: {x: 0.0, y: 0.0, z: 0.0}}")
```

publishing and latching message for 3.0 seconds

Symulacja turtlesim

Dostępne ROS Topics generowane przez turtlesim_node.

Dla pojedynczego utworzonego robota w przestrzeni nazw na przykładzie turtle1 dostępne są następujące ROS Topics:

- /turtle1/cmd_vel - prędkości sterujące robotem
- /turtle1/color_sensor - kolor
- /turtle1/pose - położenie robota

Uruchomienie symulacji:

roslaunch *turtlesim turtlesim_node*

Sterowanie manualne z klawiatury dostępne jest tylko dla robota o nazwie turtle1 i uruchamia je komenda:

roslaunch *turtlesim turtle_teleop_key*

Publisher - Python

Podstawową biblioteką do obsługi ROS w Pythonie jest **rospy**. Importowanie wiadomości na podstawie informacji o typie wiadomości jest następujące:

```
from nazwa_paczki.msg import nazwa_typu_wiadomości
```

```
In [12]: import rospy
from geometry_msgs.msg import Twist
```

Inicjalizacja node'a, aby ROS mógł jednoznacznie rozpoznać node'a.

Uwaga

1 init_node wywoływany w danym zeszycie od Jupyter Notebook.

```
In [13]: rospy.init_node("unikalna_nazwa_noda", anonymous=True)
```

Do utworzenia obiektu publishera wykorzystywana jest klasa *Publisher* z biblioteki *rospy*. Przyjmowane kolejno argumenty:

- nazwa topic'a (dla już istniejącego w systemie wykorzystywanego przez Subscriber'a lub nowa nazwa)
- typ wiadomości,
- liczba zakolejkowanych wiadomości

```
rospy.Publisher(nazwa_topicu, nazwa_typu_wiadomości, rozmiar kolejki wiadomości)
```

```
In [14]: # Przykładowy Publisher do wysyłania prędkości dla robota o nazwie turtle1 w symulacji.
pub_speed=rospy.Publisher("/turtle1/cmd_vel",Twist,queue_size=10)
```

Utworzenie i uzupełnienie wiadomości.

```
In [15]: msg = Twist()
msg.linear.x = 0.6
msg.angular.z = 1
```

Do wysłania wiadomości do robota *turtle1* jest metoda klasy *Publisher* o nazwie *publish*, która jako argument przyjmuje typ oczekiwanej wiadomości.

```
In [16]: pub_speed.publish(msg)
```

W przypadku wysyłania prędkości wysyłanie wartości prędkości z wysoką częstotliwością spowoduje, że wartości będą się bardzo szybko zmieniały i robot będzie reagował na ostatnio wysłaną wartość. Pojedyncze wysłanie prędkości powoduje, że robot wykonuje ruch zadaną prędkością około 3s.

```
In [17]: # Przykładowy ruch robota, gdy komendy wysyłane są bez przerw
msg.linear.x = 0.6
msg.angular.z = 0
pub_speed.publish(msg)

msg.linear.x = 0
msg.angular.z = 0.5
pub_speed.publish(msg)

msg.linear.x = 0.2
msg.angular.z = 0
pub_speed.publish(msg)

msg.linear.x = 0
msg.angular.z = 0.3
pub_speed.publish(msg)
```

Do wykonania przerw pomiędzy kolejnymi ruchami robota można wykorzystać opóźnienie stosując `time.sleep` z biblioteki `time`.

import time

time.sleep(czas_w_sekundach)

Zadaniem funkcji jest oczekiwanie określonego czasu przed wykonaniem kolejnej akcji

```
In [18]: # Przykładowy ruch robota - dodanie opóźnień w ruchu
import time
msg.linear.x = 0.6
msg.angular.z = 0
pub_speed.publish(msg)
time.sleep(1)

msg.linear.x = 0
msg.angular.z = 0.5
```

```
pub_speed.publish(msg)
time.sleep(1)

msg.linear.x = 0.2
msg.angular.z = 0
pub_speed.publish(msg)
time.sleep(1)

msg.linear.x = 0
msg.angular.z = 0.3
pub_speed.publish(msg)
time.sleep(1)
```

Subscriber - Python

Odpowiada za odbieranie danych pojawiających się na danym topicu.

```
In [ ]: # restart symulacji
!rosservice call reset
```

```
In [19]: # Publisher do testów susbscribera.
from geometry_msgs.msg import Twist
vel_publisher = rospy.Publisher("robot_vel", Twist, queue_size=10)
```

```
In [23]: # Utworzenie i wysłanie wiadomości do testów susbscribera.
vel_msg = Twist()
vel_msg.linear.x = 0.1
vel_msg.angular.z = 1
vel_publisher.publish(vel_msg)
```

Subscriber - prędkość postępową 0.1 i obrotową 0.3

Do utworzenia obiektu subscribera wykorzystywana jest klasa *Subscriber* z biblioteki *rospy*. Przyjmowane kolejno argumenty:

- nazwa topica (istniejąca w systemie lub nowa dla wiadomości, które będą odbierane w przyszłości)
- typ wiadomości,
- nazwa funkcji, która jest wywoływana po pojawieniu się nowej wiadomości na podanym topicu.

rospy.Subscriber(nazwa_topicu, nazwa_typu_wiadomości, nazwa_funkcji_callback)

Funkcja callback wykonuje się wtedy, gdy na topicu o podanej nazwie pojawia się nowa wiadomość. Jako argument do funkcji przekazywana jest wiadomość zgodna ze zdefiniowanym typem. (Typy wiadomości muszą się zgadzać i należy tego dopilnować.)

```
In [21]: def callback_function(msg_data):
# msg_data jest typu geometry_msgs/Twist; Jest to dokładnie wiadomość tego samego typu i pola
# zawierają te same wartości co dla wiadomości 'vel_msg' utworzonej dla Publisher'a.
print("Subscriber - prędkość postępową {} i obrotową {}".format(msg_data.linear.x, msg_data.angular.z))
```

```
In [22]: from geometry_msgs.msg import Twist
vel_subscriber = rospy.Subscriber("robot_vel", Twist, callback_function)
```

Uwaga

Wyłączenie subscriber'a.

```
In [ ]: vel_subscriber.unregister()
```

Subscriber dla położenia robota o nazwie turtle1 z symulacji.

```
In [24]: # Sprawdzenie typu wiadomości na topicu od położenia
!rostopic type /turtle1/pose

turtlesim/Pose
```

```
In [25]: # Wyświetlenie struktury wiadomości od położenia robota w symulacji
!rosmmsg show turtlesim/Pose

float32 x
float32 y
float32 theta
float32 linear_velocity
float32 angular_velocity
```

```
In [26]: from turtlesim.msg import Pose

robot_pose = Pose()
```

```
def callback_turtle1_pose(msg):
    # msg_data jest typu turtlesim/Pose;
    # zapisanie aktualnego położenia robota do zmiennej globalnej robot_pose
    global robot_pose
    robot_pose = msg
```

```
In [27]: pose_subscriber = rospy.Subscriber("/turtle1/pose", Pose, callback_turtle1_pose)
```

```
In [28]: print("Aktualne położenie robota ({:.2f},{:.2f}) oraz orientacja {:.2f}rad".format(robot_pose.x,
                                                                                          robot_pose.y,
                                                                                          robot_pose.theta))
```

Aktualne położenie robota (8.17,6.61) oraz orientacja 2.12rad

Jednoczesny Publisher Subscriber - przykład

```
In [ ]: # Sterowanie ruchem robota turtle1 w zależności od jego aktualnego położenia.
        # Subscriber odbiera informację o położeniu i je analizuje. W funkcji pojawia się wysłanie wiadomości
        # sterujących prędkością robota z użyciem Publishera.
        # Przykładowy kompletny kod obsługujący realizację zadania. Uruchomić przy zrestartowanej
        # symulacji turtlesim.
```

```
import rospy
from turtlesim.msg import Pose
from geometry_msgs.msg import Twist
rospy.init_node("pub_sub_turtlesim", anonymous=True)

pub_velocity = rospy.Publisher("/turtle1/cmd_vel", Twist, queue_size=10)

direction_right = True
def robot_control(message):
    """Analiza wiadomości i wysłanie jej na innym topicu"""
    global direction_right
    vel_msg = Twist()
    if direction_right:
        vel_msg.linear.x = 0.5
        vel_msg.angular.z = 0
    else:
        vel_msg.linear.x = -0.5
        vel_msg.angular.z = 0

    if message.x > 7:
        direction_right = False
    elif message.x < 2:
        direction_right = True
    # wysłanie przeanalizowanych danych
    pub_velocity.publish(vel_msg)

subscriber = rospy.Subscriber("/turtle1/pose", Pose, robot_control)
```

```
In [ ]: # Wersja 2 - z wykorzystaniem klasy
        # Kompletny kod.
import rospy
from turtlesim.msg import Pose
from geometry_msgs.msg import Twist
rospy.init_node("pub_sub_turtlesim_v2", anonymous=True)

class RobotMovement:
    def __init__(self):
        self.pub_velocity = rospy.Publisher("/turtle1/cmd_vel", Twist, queue_size=10)
        self.direction_right = True
        # W subscriberze funkcja callback wskazuje na metodę w klasie
        self.pose_subscriber = rospy.Subscriber("/turtle1/pose", Pose, self.robot_control)

    def robot_control(self, message):
        """Analiza wiadomości i wysłanie jej na innym topicu.
        Uzunięcie zmiennej globalnej direction_right i zastąpienie jej atrybutem klasy.
        """
        vel_msg = Twist()
        if self.direction_right:
            vel_msg.linear.x = 0.5
            vel_msg.angular.z = 0
        else:
            vel_msg.linear.x = -0.5
            vel_msg.angular.z = 0

        if message.x > 7:
            self.direction_right = False
        elif message.x < 2:
            self.direction_right = True
        # wysłanie przeanalizowanych danych
```

```
        self.pub_velocity.publish(vel_msg)

    def start(self):
        # Ponowna inicjalizacja obiektu subscriera.
        self.pose_subscriber = rospy.Subscriber("/turtle1/pose", Pose, self.robot_control)

    def stop(self):
        # Zatrzymanie odczytu informacji o położeniu.
        self.pose_subscriber.unregister()

turtle1_movement = RobotMovement()
```

```
In [ ]: turtle1_movement.stop()
```

```
In [ ]: turtle1_movement.start()
```